1495 UNIVERSITY OF ABERDEEN | CELEBRATING 525 YEARS 1495 – 2020

ABERDEEN 2040

Association Rule Mining – 1

Data Mining & Visualisation

Lecture 19

2025

Today…

- Basic concepts
- Apriori algorithm
- Different data formats for mining
- Mining with multiple minimum supports
- Mining class association rules
- Summary

Association Rule Mining

Association Rule Mining

Association rule mining is a method for understanding and identifying associations and relationships within data.

The general idea is that, given a set of transactions, we can identify groups of items that frequently occur together.

We can then generate *rules* that describe likely relationships and dependencies between different items within the dataset.

Association Rule Mining

The most common application example of association rule mining is within ‘market basket analysis’:

E.g., if someone buys a loaf of bread, what is the likelihood that they will also buy a carton of milk?

However, it also has relevance within recommender systems, and for anomaly or intrusion detection.

Definitions

Association Rule Mining – Definitions

Let’s outline some common definitions:

- An item: i
  - An item/article/object/value

E.g. an item that is sold within a store:

  - Bread, milk, cereal, etc.

Association Rule Mining – Definitions

Let’s outline some common definitions:

- An itemset
  - A set of items. Not necessarily exhaustive
  - A K-itemset is an itemset with K items

E.g. could be a set of items that are regularly sold together:

- {bread, butter, jam} is a 3-itemset
- {cereal, milk} is a 2-itemset, etc.

Association Rule Mining – Definitions

Let's outline some common definitions:

- The universal itemset: I
  - A set of all possible items sold within our store
  - $I = \{i_1, i_2, i_3, \ldots, i_n\}$

Association Rule Mining – Definitions

Let's outline some common definitions:

- Transaction: t
  - A set of items grouped together within a single 'event'
  - $t \subseteq I$

E.g. items purchased in a basket, or shopping session, within that store:

- Think of a transaction receipt that you receive after purchasing items.

Association Rule Mining – Definitions

Let's outline some common definitions:

- Transaction dataset: T
  - A set of all recorded transactions
  - $T = \{t_1, t_2, t_3, \ldots, t_n\}$

E.g. all transactions that this store has ever recorded.

  - Their transaction database, or records.

Association Rule Mining – Examples

So, for a supermarket, their Transaction set might be:

t1:	{bread, cheese, milk}
t2:	{apple, eggs, salt, yogurt}
t3:	{bread, apple, salt, milk}
t4:	{bread, cheese, yogurt}
t5:	{eggs, salt, yogurt}
…	…
tn:	{biscuit, eggs, milk}

Association Rule Mining – Examples

So, for a supermarket, their Transaction set might be:

t1:	{bread, cheese, milk}
t2:	{apple, eggs, salt, yogurt}
t3:	{bread, apple, salt, milk}
t4:	{bread, cheese, yogurt}
t5:	{eggs, salt, yogurt}
…	…
tn:	{biscuit, eggs, milk}

Here:

- bread is an item
- t1 is a transaction, which itself is an itemset
- t1 .. tn is T; our transaction dataset

Association Rule Mining – Examples

For a transaction set of text documents, we might treat each transaction (document) as a “bag” of keywords:

doc1:	Student, Teach, School
doc2:	Student, School
doc3:	Teach, School, City, Game
doc4:	Baseball, Basketball
doc5:	Basketball, Player, Spectator
doc6:	Baseball, Coach, Game, Team
doc7:	Basketball, Team, City, Game

Association Rule Mining

Note that in association rule mining, we don't care about item quantities within transactions.

E.g. if a transaction contains bread three times, we treat it exactly the same as if that transaction contained bread only once.

We also don't care about price, value, or any other properties of the transactions or of the items.

Each item is either *in a transaction*, or *it is not*.

Association Rule

An **association rule** is a rule of the form:

bread $\Longrightarrow$ milk

They capture patterns of co-occurrence in the data. I.e., if someone buys bread, they are also likely to buy milk.

Importantly, rules are one-way, meaning that:

bread $\Longrightarrow$ milk != milk $\Longrightarrow$ bread

Support and Confidence

Support and Confidence are both rule strength measures.

They are both quantitative metrics that we use during the process of creating and evaluating association rules.

We can use these to measure how common and strong our association rules turn out to be.

Support

The **support** of an itemset is the proportion of transactions within the dataset that contain that itemset.

In other words, it tells us how frequently the itemset appears in T.

$$Support = \frac{X \cap Y.count}{n}$$

…where X and Y are the items in our itemset, and n is the number of transactions in T.

Support

So if we had this Transaction dataset:

t1:	{bread, cheese, milk}	←
t2:	{apple, eggs, salt, yogurt}	
t3:	{bread, apple, salt, milk}	
t4:	{bread, cheese, yogurt}	←
t5:	{eggs, salt, yogurt}	

…the Support of {bread, cheese} would be: 2 / 5 = 0.4

Confidence

The **confidence** of a rule measures the likelihood that a transaction containing one item also contains another.

In other words, it tells us the strength of the association within that rule.

$$Confidence(X \Rightarrow Y) = \frac{Supp(X \cap Y)}{Supp(X)}$$

...where $\mathrm{X} \Longrightarrow \mathrm{Y}$ is our rule.

Confidence

So if we had this Transaction dataset:

t1:	{bread, cheese, milk}	←
t2:	{apple, eggs, salt, yogurt}	
t3:	{bread, apple, salt, milk}	←
t4:	{bread, cheese, yogurt}	←
t5:	{eggs, salt, yogurt}	

…the Support of {bread, cheese} would be: 2 / 5 = 0.4

…the Support of {bread} would be: 3 / 5 = 0.6

The Confidence of the rule:

bread ⟹ cheese

= 0.4 / 0.6 = 0.67

Association Rule Mining

Association rule mining is the process of finding all of the association rules that satisfy a user-specified **minimum support** (minsup) and **minimum confidence** (minconf).

There are many different approaches and algorithms for mining rules (which will all result in the same ruleset), but we will focus on just one:

The Apriori Algorithm

The Apriori Algorithm

The Apriori Algorithm

The Apriori Algorithm is probably the best known algorithm for association rule mining.

It has three steps:

1. Find all ‘frequent itemsets’, i.e. those which have a minimum support (≥ *minsup*);
2. Use these frequent itemsets to generate *candidate* association rules;
3. Keep only those rules which have a minimum level of confidence (≥ *minconf*)

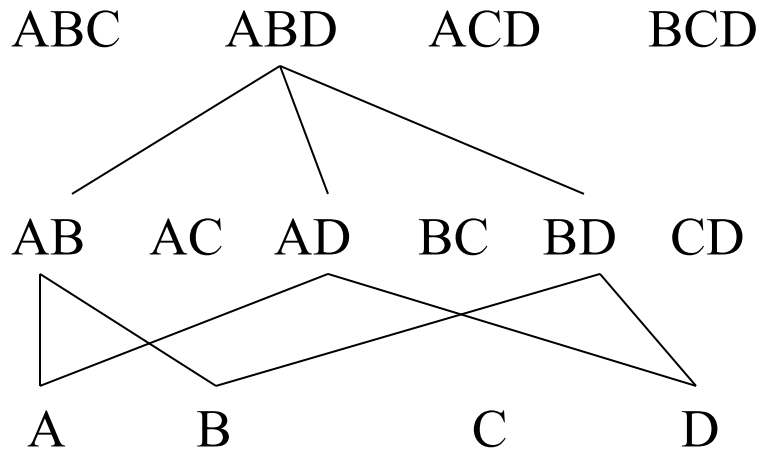
The Apriori Principle

The key idea behind the Apriori Algorithm is what's known as the **Apriori principle**, or the **downward closure property**.

This principle states that:

Any subsets of a frequent itemset are also frequent itemsets.

The Apriori Principle



The **Apriori principle** states that:

Any subsets of a frequent itemset are also frequent itemsets.

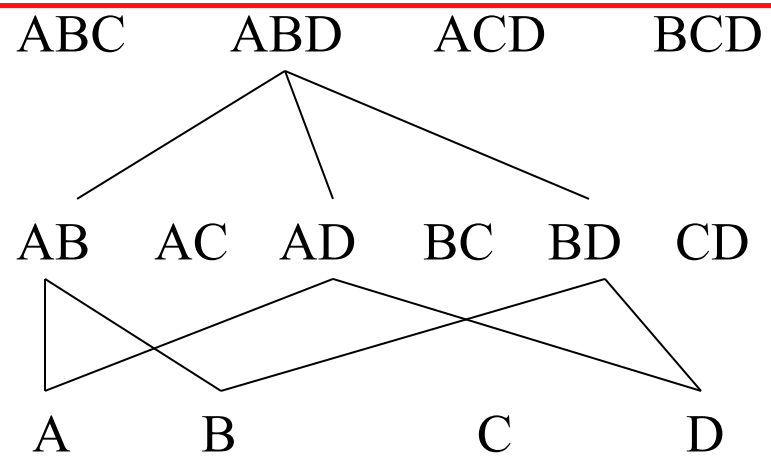
This means, for example, that if **{A, B, D}** is a frequent itemset,

then all of its non-empty subsets:

{A, B}, {A, D}, {B, D}, {A}, {B}, & {D}

are also frequent itemsets.

The Apriori Principle



Conversely:

Any supersets of an infrequent itemset are also infrequent itemsets.

This means, for example, that if **{C}** is an infrequent itemset,

then all of its supersets:

{A, C}, {B, C}, {C, D}, {A, B, C}, {A, C, D}, & {B, C, D}

are also frequent itemsets.

The Apriori Principle

The Apriori principle saves us time, given that we can effectively prune a large number of impossible itemsets (and candidate association rules) early on in the process.

This principle is the foundation of the Apriori algorithm, in which we evaluate larger itemsets, step-by-step, from smaller **frequent** itemsets.

The Apriori Algorithm

The Apriori algorithm is an iterative, level-wise search. I.e. we find all frequent 1-itemsets, then all frequent 2-itemsets, and so on.

In each iteration, k, we only consider itemsets that contain some frequent itemset from k-1.

The Apriori Algorithm

The Apriori algorithm:

1: Find all frequent 1-itemsets: F_1

2: Repeat from k = 2 onwards:

3: Generate candidate k-itemsets (C_k) from frequent (k-1)-itemsets (F_{k-1})

4: For each transaction, count how many candidates (from C_k) are contained within the transaction

5: Keep only candidates in C_k that meet *minsup*. This becomes F_k

6: Return all frequent itemsets found (F_1, F_2, F_3, …, F_k)

How Pruning Works

During the candidate generation process:

- We initially evaluate every item in I (i.e. all 1-itemsets) against the minimum support threshold.
- When a candidate 1-itemset does not meet the required level of support to be considered frequent, it is eliminated (pruned).

How Pruning Works

At each subsequent step:

- Frequent (k-1)-itemsets (i.e. those that pass the minimum support threshold) are then combined to form k-itemsets.
- We evaluate every k-itemset against the minimum support threshold.
- When a candidate k-itemset does not meet the required level of support to be considered frequent, it is eliminated (pruned).
- This process repeats until we can no longer make any itemsets.

Generating Rules From Frequent Itemsets

One last step is needed: The generation of our association rules.

For each frequent itemset where $k \geq 2$, we find combinations of items.

If A is a non-empty subset of the frequent itemset X, and B is X-A, then:

$A \Longrightarrow B$ and $B \Longrightarrow A$ can both become candidate association rules.

Note that we cannot make any association rules from 1-itemsets.

Generating Rules From Frequent Itemsets

Example: Suppose {A, B, C} is a frequent 3-itemset.

From this, we can make the following *candidate* association rules:

- A, B $\Longrightarrow$ C
- A, C $\Longrightarrow$ B
- B, C $\Longrightarrow$ A
- A $\Longrightarrow$ B, C
- B $\Longrightarrow$ A, C
- C $\Longrightarrow$ A, B

Generating Rules From Frequent Itemsets

However, since {A, B, C} is frequent, we also know that {A, B}, {A, C}, and {B, C} are also all frequent (because of the Apriori principle).

This means that we can also make these *candidate* association rules:

- $A \Longrightarrow B$
- $A \Longrightarrow C$
- $B \Longrightarrow C$
- $B \Longrightarrow A$
- $C \Longrightarrow A$
- $C \Longrightarrow B$

Generating Rules From Frequent Itemsets

For each candidate association rule, we evaluate its confidence. If its confidence ≥ minconf, then it is a valid association rule.

- A, B $\Longrightarrow$ C
- A, C $\Longrightarrow$ B
- B, C $\Longrightarrow$ A
- C $\Longrightarrow$ A, B
- B $\Longrightarrow$ A, C
- A $\Longrightarrow$ B, C
- A $\Longrightarrow$ B
- A $\Longrightarrow$ C
- B $\Longrightarrow$ C
- B $\Longrightarrow$ A
- C $\Longrightarrow$ A
- C $\Longrightarrow$ B

Reminder: $Confidence(X \Rightarrow Y) = \frac{Supp(X \cap Y)}{Supp(X)}$

Generating Rules From Frequent Itemsets

At this point, all of the required information for calculating confidence has already been recorded during the process of itemset generation.

We no longer need to see the transaction data T anymore.